Robust Variable Selection: The Experiment Plan

18 October 2026 Design draft; experiments have not been run

What we want to find out

A record may have dozens of measurements, but we can keep only a few. Can the algorithm find useful variables that still separate two classes, at a worthwhile search cost?

Our method selects variables from labelled samples. Mistakes incur a loss. Wasserstein robust selection minimizes the largest loss under specified changes to the training distribution; this objective is robust risk. Every method selects equally many variables and uses the same prediction rule, keeping selection and prediction effects separate.

There are three main experiments. Ablations change one setting to reveal its effect; we include small radius and initialization checks.

Experiment	Purpose and comparison	Main measurements
E1 Selection	Compare class information retained by a few variables on shared synthetic and public data	Test error; signal recovery
E2 Distribution changes	Fix training results; change only the test distribution	Error versus change strength
E3 Search cost	Compare broader search with stepwise selection: what does extra time achieve?	Training risk; possible improvement; time; solve count

First generate a labelled table of numbers

Each synthetic row is a record of numerical columns, with class label $k = 0, 1$. Of D columns, s carry class information; n is the training count per class. Only d columns may be selected: the observation budget d differs from the true signal count s.

For each repetition:

1. Independently draw n training records per class, with s signal values per record using the next page's rules.
2. Append $D - s$ independent standard normal noise values (mean zero, standard deviation one), independent of class and signals.
3. Independently draw twenty validation and ten thousand test records per class by the same rules. No records are copied between sets.
4. Randomly permute columns identically across all sets. Methods share these data; the next repetition draws new samples and a new permutation.

At $D = 50, n = 20, s = 4$, training has forty rows (twenty per class) and fifty columns: four signals and forty-six noise values per row. Validation also has forty rows and fifty columns; testing has twenty thousand rows and fifty columns. Labels stay with records and are never candidate variables.

Training selects variables and fits the test; validation selects parameters; testing evaluates it. Thus $n = 20$ needs twenty additional validation labels per class. Budgets reuse these data, changing only the allowed column count.

How to draw the signal values in one record

All $Z_1, \ldots, Z_s$ below are independent standard normal numbers, redrawn for every record. The class label is k. Parameters follow the common difficulty rule below.

- Different means: draw $X_j = (2k - 1)a + Z_j$ for every signal coordinate. Subtract a from the noise in class zero and add a in class one.
- Different spreads: use $X_j = Z_j$ in class zero and $X_j = rZ_j$ in class one. The centres agree, while the second class has r times the standard deviation.
- Different correlations: generate each signal pair independently. Draw two noise values and set $X_{2h-1} = Z_{2h-1}$ and $X_{2h} = c_k Z_{2h-1} + \sqrt{1 - \rho^2} Z_{2h}$, where $c_k = (1 - 2k)\rho$. Correlations have opposite signs across classes, while each column has the same marginal distribution.
- Different mixtures: independently draw a sign $B = \pm 1$ with equal probabilities for each record, and set $X_{1:s} = Ba\, v_k + Z$. Every entry of v_0 is one; the first half of v_1 is one and its second half is minus one. Here s is even and $Z = (Z_1, \ldots, Z_s)$. Individual columns have identical distributions across classes; their joint relationships differ.

All four models are calibrated to the same population difficulty: an ideal classifier that knows the true distributions and observes all s signals has a class-balanced error of 20%. Solve and save the parameters before running methods; do not adjust them after seeing results. The table gives approximations; generation saves full precision. The two a columns belong to different models.

s	Mean a	Spread r	Correlation ρ	Mixture a
2	0.595116	2.538656	0.809017	1.212285
4	0.420811	1.871151	0.600000	0.857215
8	0.297558	1.540038	0.425719	0.606142

For example, with four mean signals, draw the first four columns independently from normals with mean about -0.421 and standard deviation one in class zero; the mean is about 0.421 in class one. Append independent noise as usual. Recalibrate for each s; the four-signal correlation cannot simply be reused.

How this relates to the source paper

Section 5.1 of the source paper generates the two classes as $X = 0.4Be + Z$ and $X = 0.4Bf + Z$: e has one hundred ones; the first fifty entries of f are one and the last fifty are minus one; Z is one-hundred-dimensional standard normal noise. It uses ten training and one thousand test records per class, over five hundred trials. It has no variable selection and no unique four-column true support.

Keep that original setting as a separate data reference. Evaluate the full-variable test error, with no support recovery metric. Still draw twenty validation records per class and use this report's prediction rule, so the numbers are not directly comparable with the paper's table. This setting is outside the common 20% difficulty grid. Our mixture is its sparse adaptation; the other three models provide additional selection checks.

How scale and budget vary

First fix $s = 4$ and compare budgets $d = 2, 4, 8$ at all three scales. These budgets are insufficient, exactly matching and generous relative to the signal count. Candidate count and training size increase together; differences between scales are not attributed to either factor alone.

Scale	Candidates D	Training per class n	Signals s	Budget d
Small	20	10	4	$2, 4, 8$
Medium	50	20	4	$2, 4, 8$
Large	100	50	4	$2, 4, 8$

At the medium scale, fix $D = 50, n = 20$. For $s = 2$, use $d = 1, 2, 4$; for $s = 8$, use $d = 4, 8, 16$. Together with the existing four-signal conditions, each signal count is tested at half, equal and twice its value. Count shared conditions only once; do not cross every factor with every other factor.

Each of the four models has thirty independent repetitions per data configuration. Five data configurations and four models give six hundred independent datasets. Three budgets per dataset give 1,800 data–budget tasks. All methods share each dataset and budget. The source-paper reference adds five hundred separate trials.

How public data are used

Use the following four datasets. The first three are small-sample tabular classification problems. The last adds hundreds of features and tests generalization to people absent from training.

Data	Full dataset	Task and purpose
WDBC	569 records, 30 variables	Distinguish benign from malignant cells using cell measurements
Ionosphere	351 records, 34 variables	Distinguish good from bad radar returns
Sonar	208 records, 60 variables	Distinguish rocks from metal using sonar frequency bands
UCI HAR	10,299 windows, 561 features, 30 people	Keep sitting and standing only; test on unseen people

Split each class of tabular data into a 60% training pool, 20% validation and the remainder for testing. Round training and validation counts down. Draw $10, 20, 40$ training samples per class from the pool, nesting smaller samples within larger samples in the same split. Use ten shared splits per dataset. Exclude ID and target fields from variables.

Split HAR by person so nearby windows from the same person cannot enter both training and testing. Use the official twenty-one training and nine test people. In each repetition, reserve six training-side people for validation; draw forty windows per class as evenly as possible across the other fifteen. Use ten repetitions and budgets $8, 16, 32$. Keep alternate windows within each continuous same-person, same-activity segment to reduce overlap. Validation and testing use all retained windows from their respective people. Aggregate errors by person, rather than by window.

For each variable, subtract the training mean and divide by the training standard deviation. Apply this saved transform to validation and test data. Remove columns constant in training and record the retained count. Weight classes equally in training and evaluation.

Who we compare with, and why

Every method selects d variables. Two comparisons ask different questions: which selection criterion works well, and which search works well for the same criterion.

Four methods search the same objective. Enumeration fits every size-d subset, giving the optimum on small problems. Global search, forward selection and forward selection with swaps compare ways to approach that value with less computation. All four share data, radius, loss and the solver for each fixed subset.

Global search is our current algorithm. It evaluates subsets and uses the information obtained to rule out choices that cannot improve the result, until the target accuracy or time limit. Initialize it with the top d variables from marginal MMD.

Forward selection starts empty and adds the variable yielding the lowest robust training risk at each step. Forward selection with swaps then tries replacing one selected variable with one unselected variable, accepting the best improvement each round. Both optimize the same objective as global search but examine a smaller range of subsets. They test whether broader search warrants its extra time. Accept at most ten swaps and stop early if none improves the result.

Other selection methods represent these approaches:

- Marginal MMD: compare the two class distributions one variable at a time and select the largest differences. MMD, maximum mean discrepancy, compares within-class and between-class sample similarities; larger differences receive higher scores.
- L1 logistic regression: fit a linear classifier with a penalty that shrinks less useful coefficients, then select the largest absolute coefficients. This represents standard sparse linear selection.
- mRMR: favour variables related to the label without duplicating information already selected. Mutual information measures these dependencies. For small samples, divide values at training tertiles into three bins and merge coincident cutpoints to avoid overly fine estimates.
- Joint MMD: compare the classes on several selected variables together. Use forward selection and single-variable swaps to find larger MMD. It can detect dependencies invisible in individual variables, providing a joint-selection criterion distinct from robust risk.
- Uniform random: draw one size-d subset with a saved random seed.

Both MMD methods use Gaussian similarity $\exp(-\|x - y\|_2^2/(2h^2))$, where h is the median positive training-sample distance on the current subset. The score is the sum of the two mean within-class similarities minus twice the mean between-class similarity; within-class averages exclude self-pairs. Joint MMD directly maximizes this score and follows the same swap rule as forward selection with swaps.

Also report all s true signals and all D variables as interpretive references: they show performance when signal locations are known or the observation budget is relaxed. Exclude them from equal-budget rankings.

How predictions remain comparable

Refit the same robust score on every selected subset. Its output lies in $[-1, 1]$, with positive values favouring class 0. The training loss is $\ell(t) = \frac{1}{2}\max(0, 1 + t)$, making each fixed-subset fit a linear program: an optimization problem with a linear objective and constraints. Use the function construction in the theory note to score a new point from its selected-coordinate distances to training points in both classes.

The robust radius controls how far a distribution may move from the training samples. A smaller radius trusts the data more; a larger radius protects against greater changes. After standardization, divide each class's median pairwise training ℓ_1 distance by D, calling this single-coordinate scale b_k. Candidate radii are $\theta_k = \rho_0 b_k$, with $\rho_0 \in \{1/8, 1/4, 1/2, 1, 2\}$. The actual transport cost remains the sum of absolute changes in selected coordinates.

At reference budget four (sixteen for HAR), obtain three subsets from marginal MMD, mRMR and joint MMD. Select one common radius pair by their average validation error across candidate radii. Fix it for all methods and budgets thereafter. In L1 logistic regression, smaller C means stronger regularization; choose from $\{0.01, 0.1, 1, 10, 100\}$ by the same validation evaluation. Resolve ties toward the larger radius, stronger penalty and smaller variable index. Test data never enter these choices.

E1: How much class information do a few variables retain?

Use the four models, three scales and three budgets above, adding $s = 2, 4, 8$ at the medium scale. Plot error and recovery against budget, rather than reporting only $d = 4$. The two-signal correlation and mixture models have no single-column class information at budget one; show this limitation.

For the first three public datasets, fix budget four and compare $10, 20, 40$ training samples per class. Then fix twenty per class and compare budgets $1, 2, 4, 8$. Run the shared centre only once: six conditions and ten splits per dataset give 180 instances. HAR compares $8, 16, 32$, giving thirty instances.

The primary predictive metric weights the two class errors equally. Convert score $f(x)$ to the probability of accepting class 0, $p_0(x) = (1 + f(x))/2$. In class 0 an error has probability $1 - p_0(x)$; in class 1 it has probability $p_0(x)$. Average within each test class, then average the classes:

$$\mathrm{BER}_{\mathrm{prob}} = \frac{1}{2}\big(\mathrm{mean}_{Y=0}(1 - p_0(X)) + \mathrm{mean}_{Y=1}\, p_0(X)\big).$$

Compute probabilities directly without additional coin tosses. Error 0.5 corresponds to a constant prediction with no class information. Training risk sums the two class losses and is twice the corresponding balanced error; optimization gaps use this sum. Also report balanced error when nonnegative scores predict class zero and negative scores predict class one. Retain each class's error separately, so improvement in one cannot hide deterioration in the other.

For synthetic data, also check signal recovery. If h selected variables are true signals, precision is h/d and recall is h/s. Undo the column permutation before computing recovery. In the mixture model, one pair spanning both halves may suffice for discrimination; complete recovery is not required for accurate prediction. For public data, report selected variables and selection frequencies.

Compare the three scales in tables. At the medium scale, plot error and signal recall against budget. Summarize public results by dataset and sample size. Report means and standard deviations over thirty synthetic repetitions. Compute a method difference within each repetition, then resample these thirty differences with replacement two thousand times for a 95% interval. For tabular data, report paired differences and standard deviations over ten splits. For HAR, average training repetitions within each test person, weight the nine people equally and resample people.

E2: Do the selected variables remain useful after a change?

Reuse the medium-scale training results for all four models at $s = d = 4$, with thirty repetitions each. Do not reselect variables or retune parameters. Make two changes to the test data: translate the four signal coordinates together, or add independent normal measurement noise to those coordinates. Every method receives exactly the same changed samples.

Measure change against the average radius $\bar{\theta} = (\theta_0 + \theta_1)/2$, using $\delta/\bar{\theta} = 0, 1/2, 1, 2$. The translation vector is $\delta(1,1,1,1)/4$. Set the noise standard deviation to $\delta/(4\sqrt{2/\pi})$, so both changes have expected total absolute displacement δ. Apply them in the saved standardized coordinates.

For each signal model, plot one error curve per method and report the increase from no change to the largest change. This asks which method degrades faster under the specified changes. The training optimization interval constrains loss within the stated Wasserstein set; it does not automatically guarantee error under arbitrary test changes.

For the radius ablation, add only twenty medium-scale mean and mixture instances at $s = d = 4$, ten each. Hold data fixed and compare $1/2, 1, 2$ times the selected radius for global search and forward selection with swaps, giving 120 method runs. These choices provide weaker protection, the original setting and stronger protection. Check error curves and constant predictions; do not use test results to reselect a radius.

E3: Is global search worth its computational cost?

First compare enumeration, global search, forward selection and forward selection with swaps on small problems. Use $D = 10, s = 4, d = 3$, fixed radii $\theta_k = b_k/4$, five or ten training samples per class, and five repetitions of each of the four models: forty instances. Enumerate all 120 size-three subsets to obtain the optimum under the common solver, then verify it with an independent fixed-subset linear program. Report excess risk above this optimum, runtime and fit count for each method.

A cut is a risk lower-bound formula valid for all subsets. Check that each cut lies below the true value on all 176 subsets of size at most three, and that the returned interval contains the enumerated optimum. Use numerical-check tolerance 10^{-8} and outer interval width at most 10^{-6}. Complete all enumeration and checks.

For larger problems, compare global search, forward selection and forward selection with swaps. At $D = 50, d = 4$, enumeration already needs $230{,}300$ subsets; it is not required to finish large tasks. Time-limited enumeration is not an optimal-value reference. Use four-signal mean and mixture models centred at $D = 50, n = 20, d = 4$. Change one factor at a time:

$$D \in \{20, 50, 100, 200\}, \qquad n \in \{10, 20, 50, 100\}, \qquad d \in \{2, 4, 8, 12\}.$$

Count the shared centre once: ten conditions per model and ten repetitions each give 200 instances. Use common radii $\theta_k = b_k/4$ to separate parameter selection from search cost. Fix the hardware and use one solver thread, with three hundred seconds per search. Time every instance once; repeat timing three times only for the twenty centre instances, reporting their median and range.

Global search must report both the solution found and how much improvement remains possible. The algorithm uses $0/1$ variables to indicate whether each column is selected, and passes them to an integer solver. A feasible solution supplies a risk upper bound U, while the solver's proven global lower bound is L. Their difference $U - L$ is the improvement not yet ruled out. The current feasible objective in the search is not a lower bound.

Plot U, L against time. Record time to first reach the final best solution, time to the target gap, the number of fixed-subset fits, and search time spent choosing subsequent subsets. Compare the risk found within three hundred seconds and the number of global-search instances certified by $10, 60, 300$ seconds. Report feasible risk and runtime for greedy methods, without assigning them a global certificate they do not provide.

Only the necessary execution rules

In statistical experiments, allow three hundred seconds for variable selection per candidate configuration, or six hundred seconds for HAR. A fixed-subset fit has at most sixty seconds and cannot exceed the remaining search budget. Use global absolute gap 10^{-3} and fixed-subset risk upper–lower gap at most 10^{-4}; use the latter as the minimum swap improvement too. Keep constraint violations, normalized by their own scales, within 10^{-7}. These numbers control solver accuracy and are not tuned for predictive performance.

Charge every method for initialization, candidate swaps and final fitting. Report common radius-selection time separately and include it in total time. At timeout, retain the best verified feasible result and its gap. If no verifiable test is available, return the constant zero score and mark failure, while retaining its predictions in summaries. If the budget is unfilled, add the remaining highest-ranked marginal-MMD variables and retain an incomplete status.

The initialization ablation reuses the twenty radius-ablation instances. At the originally selected radius, replace MMD initialization for global search with one random size-d subset, adding only twenty runs. Compare solution quality, solve count, time and final gap.

Retain data versions, splits, seeds, model parameters, standardization rules, selected subsets, validation choices, errors, solver statuses and time traces. First check implementation on ten development instances excluded from final statistics; freeze the rules before running these comparisons. All numbers in this note are planned settings. No results or method rankings are filled in.